

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 10(B)
Linked List Exercises

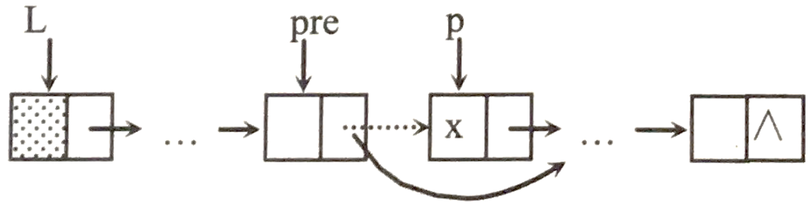
Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Exercise 1

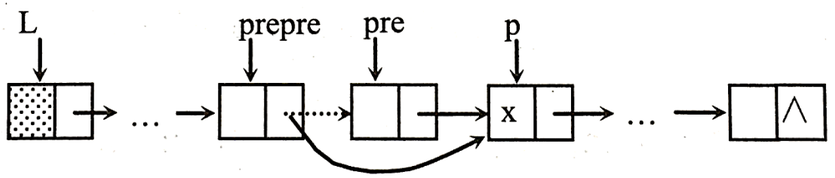
- 例子：设计一个算法删除单链表L中第一个值为x的节点。
- 思路：如图所示，用**pre, p**遍历整个单链表，pre指向*p的前驱结点，p用于查找第一个值为x的结点。当找到后将*p结点删除，返回1；否则返回0。对应算法如下。




```
int delx(LinkList *&L,ElemType x)
{   LinkList *pre=L,*p=pre->next;          //pre 指向*p 的前驱节点
    while (p!=NULL && p->data!=x)
    {   pre=p;
        p=p->next;                         //pre、p 同步后移一个节点
    }
    if (p!=NULL)                           //找到值为 x 的*p 节点
    {   pre->next=p->next;
        free(p);
        return 1;
    }
    else return 0;                         //未找到值为 x 的节点
}
```

Exercise 2

- 例子：设计一个算法，删除单链表L含两个或两个以上的数据结点中，第一个值为x的结点的前驱结点。



- 思路：如图所示，在遍历时用p指向值为x的结点，用pre指向*p结点的前驱结点，用prepre指向*pre节点的前驱结点。若第一个数据结点的值为x或未找到值为x的结点，返回0；否则删除*pre结点，并返回1。对应的算法实现如下：

查找

删除

```
int delx1(LinkList *&L,ElemType x)
{   LinkList *prepre=L,*pre=prepre->next,*p;
    if (pre->data==x) return 0;          //第一个数据节点值为 x,返回 0;
    p=pre->next;
    while (p!=NULL && p->data!=x)        //查找值为 x 的*p 节点
    {   prepre=pre;
        pre=p;
        p=p->next;                       //prepre、pre、p 同步后移一个节点
    }
    if (p!=NULL)                         //找到值为 x 的*p 节点
    {   prepre->next=p;                  //删除*pre 节点
        free(pre);                       //释放*pre 节点空间
        return 1;                        //成功删除返回 1
    }
    else return 0;                       //未找到值为 x 的节点,返回 0
}
```

Exercise 3

- 例子：在**带头结点**的单链表中，删除所有值为x的结点，并释放其空间。假设值为x的结点不唯一，试编写算法实现以上的操作。
- **思路1：**用p从头到尾扫描单链表，pre指向*p结点的前驱。若p所指结点的值为x，则删除，并让p移向下一个结点，否则让pre, p指针同步后移一个结点。

```
void Del_X_1(Linklist &L,ElemType x){
//L 为带头结点的单链表，本算法删除 L 中所有值为 x 的结点
    LNode *p=L->next,*pre=L,*q; //置 p 和 pre 的初始值
    while(p!=NULL){
        if(p->data==x){
            q=p;                    //q 指向该结点
            p=p->next;
            pre->next=p;            //删除*q 结点
            free(q);                //释放*q 结点的空间
        }
        else{                       //否则，pre 和 p 同步后移
            pre=p;
            p=p->next;
        }//else
    }//while
}
```

本算法是在**无序单链表**中删除满足某种条件的所有结点，这里的条件是结点的值为x。

实际上，**这个条件是可以任意指定的**，只要修改if条件即可，比如，要求删除介于mink和maxk之间的所有结点，则只需将if语句修改为：

if(p->data > mink && p->data < maxk)。

Exercise 3

- 例子：在**带头结点**的单链表中，删除所有值为x的结点，并释放其空间。假设值为x的结点不唯一，试编写算法实现以上的操作。
- **思路2：**采用尾插法建立单链表。用p指针扫描L的所有结点，当其值不为x时，将其链接到L之后，否则将其释放。

```
void Del_X_2(Linklist &L,ElemType x){
//L为带头结点的单链表，本算法删除L中所有值为x的结点
    LNode *p=L->next,*r=L,*q;    //r指向尾结点，其初值为头结点
    while(p!=NULL){
        if(p->data!=x){          //*p结点值不为x时将其链接到L尾部
            r->next=p;
            r=p;
            p=p->next;           //继续扫描
        }
        else{                    //*p结点值为x时将其释放
            q=p;
            p=p->next;           //继续扫描
            free(q);             //释放空间
        }
    }//while
    r->next=NULL;                //插入结束后置尾结点指针为NULL
}
```

两种算法皆扫描一遍链表，

时间复杂度为O(n)

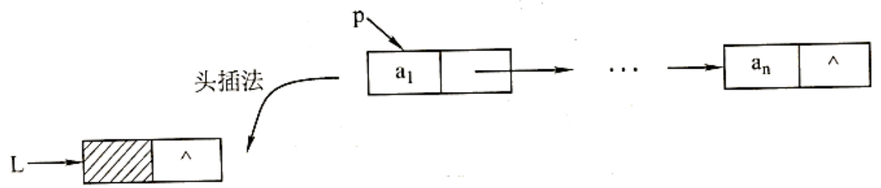
空间复杂度为O(1)

Exercise 4

- 例子：试编写算法将带有头结点的单链表就地逆置，所谓“就地”是指辅助空间复杂度为O(1)。
- 思路1：将头结点摘下，然后从第一结点开始，依次前插入到头结点的后面（头插法建立单链表），直到最后一个结点为止，则实现了链表的逆置，如图所示。

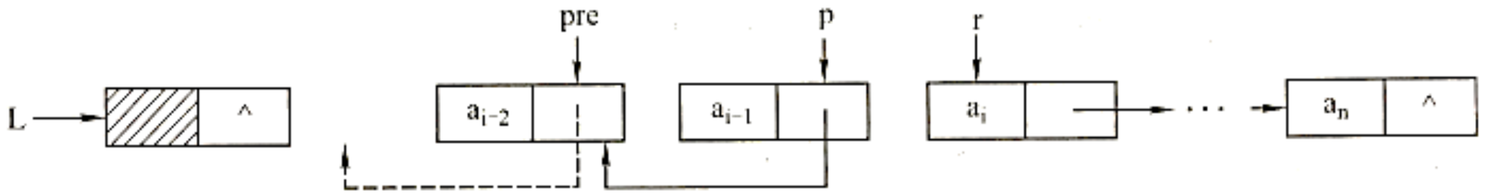
```
LinkList Reverse_1(LinkList L){
//L是带头结点的单链表，本算法将L就地逆置

    LNode *p, *r;                  //p为工作指针，r为p的后继，以防断链
    p=L->next;                     //从第一个元素结点开始
    L->next=NULL;                  //先将头结点L的next域置为NULL
    while(p!=NULL){                //依次将元素结点摘下
        r=p->next;                 //暂存p的后继
        p->next=L->next;           //将p结点插入到头结点之后
        L->next=p;
        p=r;
    }
    return  L;
}
```



Exercise 4

- 例子：试编写算法将带有头结点的单链表就地逆置，所谓“就地”是指辅助空间复杂度为O(1)。
- 思路2：假设 pre, p 和 r 指向3个相邻的结点，如图所示。



- 假设经过若干操作，*pre之前的结点的指针都已经调整完毕，它们的next都指向其原前驱结点。现在令*p结点的next域指向*pre结点，注意到一旦调整指针的指向后，*p的后继结点的链接就断开了，为此需要用r来指向原*p的后继结点。
- 处理时需要注意两点：
  - 一是在处理第一个结点时，应将其next域置为NULL，而不是指向头结点（因为它将作为新表的尾结点）；
  - 二是在处理完最后一个结点后，需要将头结点的指针指向它。

Exercise 4

- 例子：试编写算法将带有头结点的单链表就地逆置，所谓“就地”是指辅助空间复杂度为O(1)。

```
LinkList Reverse_2(LinkList L){
//依次遍历线性表 L，并将结点指针反转
    LNode *pre,*p=L->next,*r=p->next;
    p->next=NULL;                    //处理第一个结点
    while(r!=NULL){                  //r 为空，则说明 p 为最后一个结点
        pre=p;                       //依次继续遍历
        p=r;
        r=r->next;
        p->next=pre;                 //指针反转
    }
    L->next=p;                       //处理最后一个结点
    return  L;
}
```

- 上述两个算法思路的时间复杂度为O(n)，空间复杂度为O(1)。

Thank you!

